TestOrCode: 5-day emergency execution plan

HUMAN1 = manoj

- has one claude code assistance

HUMAN 2 + HUMAN3 = charan + whoever is free

- has one claude code assistance

October 25–29, 2026 · Mid-submission: October 30 · Team: 3 humans + 2 Claude Code instances

Your situation is urgent, but manageable if you stop treating the next five days as a normal development sprint.

The objective is to have a demonstrable MVP, real preliminary experimental results, a reproducible GitHub repository, and a credible 15-day completion plan by October 29. October 30 should be for submission, not for finishing the implementation.

Your mid-submission email explicitly requires:

- Progress and the current datasets.
- Baseline experiments, ablations and results from experiments actually run.
- Challenges faced and mitigation plans.
- A timeline for final submission.
- An updated GitHub repository containing all code.

Your submitted TestOrCode proposal describes four logical agents, intent-grounded retrieval, persistent memory with executable probes, test-integrity checking, three decision outcomes, a 72-episode evaluation set and a B1 baseline with three core ablations. The course specification expects a research-oriented system with a controlled evaluation and quantitative evidence, rather than a demo alone.

With only five days, do not attempt to complete all 504 proposed episode-runs. Build the smallest honest implementation of the research idea, execute a small pilot, and demonstrate that the remaining work is technically and experimentally feasible.

1. Define the minimum viable submission

I would organize the work into three levels.

Level 1 — Must have by October 30

- A working GitHub repository with setup instructions.
- An end-to-end TestOrCode prototype.
- A dataset manifest with genuine, traceable examples and known labels.
- One functioning B1 baseline.
- All three ablation configurations implemented or clearly identified as incomplete, with any completed runs reported.
- Real preliminary results from executed experiments.
- A mid-submission report covering progress, challenges, mitigations and the final 15-day plan.
- A short recorded or live demonstration.

Level 2 — Simplify aggressively

- Use one Python orchestrator with four role-specific prompts instead of four independent agent frameworks.
- Start with one repository and 6–12 manually verified pilot episodes.
- Use BM25 retrieval first; add embeddings if time permits.
- Use a simple JSON memory store and executable probes.
- Start with cached mutants and deterministic test-integrity checks.
- Use a CLI for the demo instead of building a frontend.

Level 3 — Explicitly defer to the following 15 days

- Scaling to all three held-out libraries and 72 episodes.

- Running the full 504-run matrix and second seeds.
- Comprehensive mutation testing across the evaluation corpus.
- Statistical significance testing and full failure analysis.
- Optional reranking, second-model comparison and fine-tuning.

One important distinction: you can simplify the implementation, but you cannot invent results or claim a component works when it has only been mocked. If an ablation has not run, report it as planned or implemented but untested. If a backend is mocked, label it as a mock. That is much more defensible during evaluation.

2. What exactly should the MVP do?

Do not build a dashboard. Build a small Python CLI that demonstrates the research question directly.

Input: a CI failure episode

Code diff · Commit message · Failing pytest output

Triage + Intent Analyst

Run tests · Retrieve evidence · Produce a structured hypothesis

Repairer

Apply a scoped patch or recommend escalation

Integrity Guard

Rerun tests · Check allowed files · Verify test strength

UPDATE_TEST

FIX_CODE

ESCALATE

The MVP should support these concrete capabilities:

1. Run pytest and capture failures.
2. Retrieve relevant commit messages or documentation using BM25.
3. Ask the model for a structured decision, with evidence references.

4. Apply a restricted patch or escalate.
5. Rerun tests and check whether the repair was safe.
6. Store an approved memory record with a probe, and reuse it only when the probe passes.
7. Save experiment results as JSON or CSV.

For the mid-evaluation, you do not need perfect autonomous behaviour across arbitrary repositories. You need a real, reproducible path through the system on representative cases.

3. Task delegation: three humans and two Claude Code instances

The key to using two coding agents effectively is to assign them separate codebases and interfaces. Do not let both instances independently build the same architecture.

Human 1

Agent orchestration and baseline

Owns the core Python application.

- Define the episode schema and decision labels.
- Implement the orchestrator and four role-specific prompts.
- Connect the hosted LLM API and structured tool calls.
- Implement the B1 single-agent baseline.
- Ensure the three outcomes are handled correctly.
- Integrate the final CLI demo.

Success by Day 3: Full and B1 both run on the same episode format.

Human 2

Sandbox, memory and integrity

Owns execution safety and verification.

- Set up the repository environment and pytest runner.

- Implement restricted patch application.
- Add checks for edits outside permitted paths and obvious test weakening.
- Implement JSON-backed memory and executable probes.
- Add a cached-mutant runner for a small number of functions.
- Implement A3, the no-Integrity-Guard ablation.

Success by Day 3: A patch can be applied, tested, checked and rejected if unsafe.

Human 3

Dataset, RAG and experiments

Owns the evidence and evaluation pipeline.

- Select a repository with a manageable test suite.
- Construct 6–12 traceable pilot episodes with verified gold labels.
- Implement BM25 retrieval over commit messages and documentation.
- Implement A1 (no intent retrieval) and A2 (no persistent memory).
- Build the experiment runner and metrics scripts.
- Prepare result tables, plots and the evaluation report.

Success by Day 3: Full, B1 and ablation configurations can be executed against the same labelled episodes.

Claude Code 1

Core application boilerplate

Assign it to Human 1's branch and interface.

- Generate the Python package structure and typed data models.
- Implement API-client wrappers, tool dispatch and structured-output validation.
- Generate role prompts and state-machine scaffolding.
- Write unit tests and fix integration errors.
- Implement the common interface used by B1 and the Full system.

Human 1 remains responsible for the architecture, decisions and code review.

Claude Code 2

Dataset and verification tooling

Assign it to Human 2/3's shared tooling branch, with separate tasks.

- Generate the pytest execution and Git utility wrappers.
- Implement restricted patch checks and integrity-check scaffolding.
- Generate episode-manifest validation and metric aggregation scripts.
- Add fixture-based unit tests for memory probes and ablation switches.
- Help debug the evaluation harness.

Human 2 owns the sandbox and integrity code; Human 3 owns dataset correctness and metrics.

Shared interface agreed on Day 1

Use one episode format and one result format across all five experimental conditions.

For example, an episode record should contain:

- `episode_id`
- `repository`, `base_commit` and `current_commit`
- `diff`, `commit_message` and `failing_tests`
- `gold_labels` and `gold_evidence_ids`

Each run should record the condition, seed, model ID, predicted labels, retrieved evidence, patch, final test status, integrity results, token usage, tool calls and error details.

Do not expose the gold labels or reference repair to the agent itself. Keep them in the evaluation harness.

Important: the hosted model should be accessed through a common client, and all conditions must use the same model and comparable budgets. Claude Code is your development tool; it is not the model being evaluated in your experiments.

4. Day-by-day action plan

Assume each day runs from 9:00 AM to 8:00 PM, with short breaks and a lunch break inside the schedule. That gives approximately 10 hours of productive work per person per day.

Day 1 — Sunday, October 25: Lock scope and make the first API call

Do not over-engineer today

Target: repository, interfaces, working model call, and an executable test environment.

09:00–11:00

Team kickoff and interface freeze

All three humans agree on the folder structure, episode schema, result schema, allowed actions and model API. Select one candidate Python library and one backup. Create the GitHub repository, branches and `.gitignore`.

11:00–13:00

Parallel scaffolding

- Human 1 + Claude Code 1: orchestrator, LLM client and structured output.
- Human 2 + Claude Code 2: test runner, patch restrictions and basic safety tests.
- Human 3: inspect the candidate repository, identify real commits and draft the pilot episode manifest.

13:00–15:00

Make the foundation real

Run the repository's tests, capture a genuine failing case, make one hosted-model API call, and confirm the agent can return a valid structured decision. Create a working development environment and lock dependencies.

15:00–18:00

Build the first vertical slice

Connect test execution → model decision → restricted patch or escalation → test rerun. Human 3 creates and verifies the first 3–4 episodes and starts the retrieval index.

18:00–20:00

Integration checkpoint

Merge working components, run smoke tests and commit everything. Document blockers and assign fixes for tomorrow.

Day 1 exit criteria

- [ ] GitHub repository is usable and has a README.
- [ ] Hosted-model call succeeds.
- [ ] Test runner can reproduce a failure.
- [ ] A single episode can flow through at least part of the system.
- [ ] Pilot dataset and interface schemas exist.

If you do not have a working model call and test runner by tonight, simplify the API integration and environment immediately.

Day 2 — Monday, October 26: Make the core system work

Target: a real Full-system path, B1 baseline and a usable labelled pilot dataset.

Time	Human 1	Human 2	Human 3
09:00–11:00	Complete orchestration and role prompts	Finish patch restrictions and test runner	Complete initial episode manifests
11:00–13:00	Implement Triage and Analyst outputs	Implement assertion/skip checks	Implement BM25 retrieval and evidence IDs
13:00–15:00	Connect Repairer and API tools	Implement memory records and probe execution	Verify labels against source/test diffs
15:00–18:00	Build B1 single-agent baseline	Add mutation-test runner for selected	Implement metrics and experiment runner

Time	Human 1	Human 2	Human 3
		functions	
18:00–20:00	Integrate and fix failures	Test unsafe patches and memory invalidation	Run first Full/B1 pilot

Claude Code 1 should focus on the agent workflow and API integration. Claude Code 2 should focus on safety checks, fixtures and the evaluation utilities.

Day 2 exit criteria

- The Full system can process at least one real episode end to end.
- B1 can execute on the same input.
- BM25 retrieves actual repository evidence.
- At least 6 labelled pilot episodes exist, with the goal of reaching 8–12 by Day 3.
- Results are saved in machine-readable form.

Do not build a web interface. A CLI that prints the decision, evidence, patch and verification results is sufficient.

Day 3 — Tuesday, October 27: Implement ablations and generate real preliminary results

Target: prove the experiment can run, not just the demo.

Time	Human 1	Human 2	Human 3
09:00–11:00	Stabilize Full/B1 shared interfaces	Implement A3 without Integrity Guard	Finalize pilot labels and gold evidence
11:00–13:00	Implement A1 switch with Member 3	Validate patch scope and memory probes	Implement A2 switch and experiment configuration
13:00–15:00	Add deterministic sanity baselines	Run integrity tests and selected mutants	Test all five conditions on 2–3 episodes
15:00–18:00	Fix common agent failures	Fix sandbox and evaluation integration	Run Full, B1, A1, A2 and A3 on the pilot
18:00–20:00	Review decisions and logs	Review integrity outcomes	Produce first results table and plots

Your five conditions are:

- Full: all four logical roles and the proposed components.
- B1: one tool-using agent, standard repository RAG and no persistent memory.
- A1: no intent retrieval.
- A2: no persistent memory.
- A3: no independent Integrity Guard.

The deterministic always-`UPDATE_TEST` and always-`FIX_CODE` baselines are inexpensive and should also be included.

The submitted proposal defines these as the three main ablations, so make them configurable from the beginning rather than writing three separate applications.

Day 3 exit criteria

- All five configurations can be invoked, even if some need more debugging.
- You have run actual experiments on a small set of labelled episodes.
- The first metrics table exists.
- Every incomplete component is explicitly recorded.
- You can explain at least one failure and one successful repair from actual logs.

If you only have 3–4 verified episodes by this point, report the actual count. Do not manufacture a 12-episode dataset just to meet a target.

Day 4 — Wednesday, October 28: Run experiments, prepare evidence and write the report

Target: preliminary results, complete mid-submission documentation and a stable demonstration candidate.

Time	Human 1 — Agent system	Human 2 — Safety	Human 3 — Evaluation
09:00–11:00	Fix Full/B1 failures	Fix sandbox and integrity bugs	Validate labels and result logs
11:00–13:00	Run Full and B1 on the frozen pilot	Run integrity checks and A3	Run A1 and A2; collect metrics

Time	Human 1 — Agent system	Human 2 — Safety	Human 3 — Evaluation
13:00–15:00	Test ambiguous and mixed cases	Test unsafe patches and probe failures	Compute macro-F1, repair rate and error counts
15:00–17:00	Document architecture and decisions	Document safety design and limitations	Draft results, dataset and baseline sections
17:00–19:00	Help integrate report and demo	Validate setup instructions and clean Git state	Complete challenges, mitigations and 15-day plan
19:00–20:00	Review report	Review report	Review report

Claude Code 1 should help produce architecture documentation and troubleshoot orchestration. Claude Code 2 should help generate report tables from the actual result files, verify the repository instructions and identify missing tests.

Do not ask either coding agent to invent results or write conclusions before seeing the data.

Day 4 exit criteria

- Every experimental result in the report can be traced to a saved run.
- The dataset table shows the actual number of episodes and their label distribution.
- The Full-versus-B1 comparison exists, even if the sample is small.
- Ablation results are reported only for completed runs.
- The challenges and mitigation sections are drafted.
- The final 15-day timeline is included.

Day 5 — Thursday, October 29: Integration, bug fixes and submission freeze

As requested, Day 5 is strictly for integration, fixing breaking bugs, final documentation and the demo. No new features.

09:00–11:00

Full integration test

Pull all approved branches, resolve merge conflicts, create a clean environment and run the smoke-test suite. Verify the CLI, model API, dataset loader, baseline and ablation switches.

11:00–13:00

Fix breaking bugs only

Fix import failures, schema mismatches, API configuration, crashes, broken tests and incorrect result logging. Claude Code instances may debug existing defects, but must not introduce new features.

13:00–15:00

Freeze and verify results

Rerun the final smoke tests and confirm that the metrics in the report match the saved JSON/CSV. Record incomplete experiments, known issues and the exact current dataset size.

15:00–17:00

Final report and GitHub

Complete the mid-submission report, update the README, commit the working code, verify setup instructions, check `.gitignore` and remove any exposed API keys.

17:00–19:00

Record and rehearse the demo

Show a real failure, retrieved evidence, the agent's decision, the repair or escalation, and the verification output. Also show a baseline or ablation comparison using the actual recorded results.

19:00–20:00

Submission freeze

Tag or identify the final commit, back up the report and result files, verify that the repository is accessible, and prepare a concise progress summary for the

evaluators.

If a feature is still broken at 5 PM, remove or disable that feature cleanly and disclose its status. Do not spend the final hour attempting a risky architectural rewrite.

5. What should the mid-submission report contain?

The email sets the required content. I would prepare a concise report of roughly 6–10 pages, with actual progress and evidence rather than a repetition of the proposal.

Suggested report structure

Tick items as they are ready for submission.

0 of 10 sections ready

0%

- [ ] Executive summary and current project status
- [ ] Research question and proposed architecture
- [ ] Implemented components versus planned components
- [ ] Dataset sources, construction process, labels and actual pilot size
- [ ] B1 baseline, three ablation definitions and completed experiments
- [ ] Preliminary quantitative results and limitations
- [ ] Challenges faced and mitigation plans

- [ ] Updated GitHub repository and reproducibility instructions
- [ ] Detailed October 31–November 14 completion plan
- [ ] Demo link and known limitations

For the preliminary results, include a table like this and fill it exclusively from real runs:

Condition	Episodes run	Direction macro-F1	Correct-repair rate	Masked regressions
Full	Actual count	Measured	Measured	Measured
B1	Actual count	Measured	Measured	Measured
A1 — No intent retrieval	Actual count	Measured	Measured	Measured
A2 — No memory	Actual count	Measured	Measured	Measured
A3 — No Integrity Guard	Actual count	Measured	Measured	Measured

Small pilot samples can yield unstable metrics. Show raw counts alongside rates, and explicitly label these as preliminary. If a condition has not run, mark it as not yet evaluated rather than presenting an empty result as zero.

6. The final 15-day plan: October 31–November 14

The mid-submission email says the final deadline is November 15, so you have 15 calendar days after October 30. Keep the work sequenced as follows.

Oct 31 – Nov 2

Stabilize dataset and environments

Finish the episode-construction scripts, select the three evaluation libraries, validate gold labels, pin dependencies and complete the separate development

stream.

Nov 3 – 5

Complete all core features

Finish hybrid RAG, probe-backed memory, Integrity Guard and fault recovery. Freeze the model and configuration using development data.

Nov 6 – 9

Run the full evaluation

Run the 72-episode test set for Full and B1 with two seeds, then A1, A2 and A3. Include deterministic sanity baselines and checkpoint every run.

Nov 10 – 11

Analyse and validate

Calculate metrics, conduct paired statistical comparisons where feasible, analyse failure causes, and investigate incorrect repairs and test weakening.

Nov 12 – 13

Report and demo

Complete the research report, results tables, plots, limitations, reproducibility package and final demonstration.

Nov 14

Final release

Run a clean setup test, check the repository, back up all results and prepare the final submission. Keep Nov 15 for submission rather than development.

One adjustment is essential: do not wait until November 6 to discover that the full experiment matrix exceeds your API budget or runtime. Benchmark several episodes on Day 3, then measure throughput again as soon as the evaluation environment stabilizes. Your proposal estimates thousands of LLM calls for the complete matrix, so the pilot must establish whether that schedule is achievable.

AgentZero.pdf

7. Rules for the next five days

1. Working code beats elegant architecture. Prefer a deterministic Python state machine with structured agent calls over a new framework.
2. No frontend. A CLI and a clear terminal demonstration are sufficient.
3. No optional fine-tuning. Your proposal already identifies it as the first feature to drop when time is short.

   LMA_idea_triage_final.pdf
4. No separate applications for ablations. Use configuration flags to switch components off.
5. No unverified benchmark claims. Every number in the report must come from logged experiments.
6. Commit every working milestone. Never wait until Day 5 to push the code.
7. Freeze features on Day 4 and freeze the release on Day 5. Only bug fixes after that.

The most important decision is to avoid spending all five days writing the agent and leaving no time for evidence. Your mid-evaluation is explicitly checking that the project is progressing scientifically as well as technically.

By October 29 at 8 PM, aim to show three things: a working end-to-end MVP, real Full-versus-B1 and preliminary ablation results, and a GitHub repository plus report that makes the remaining 15 days look like a credible completion plan rather than an unstarted project.

GIT, Tech Stack, Models

1. Git branching and assignment strategy

Recommendation: 4 long-lived branches maximum

Use a lightweight feature-branch workflow with frequent merges into `main`. Do not create a separate branch for every human and Claude Code instance.

`main`

Always runnable · Integrated and tested code only

`feat/agent-core`

Human 1

Orchestrator, model client, baselines

`feat/sandbox`

Human 2

Pytest, patches, memory, integrity

`feat/evaluation`

Human 3

Dataset, retrieval, metrics, ablations

The two Claude Code instances work under human supervision on the appropriate feature branches. They do not need their own permanent branches.

Exact assignment

Owner	Branch	Primary files and responsibilities
Human 1	`feat/agent-core`	Orchestrator, agent prompts, LLM API client, tool dispatch, B1 baseline
Human 2	`feat/sandbox`	Docker, pytest execution, Git tools, patch restrictions, memory, Integrity Guard, A3
Human 3	`feat/evaluation`	Episode dataset, BM25 retrieval, A1/A2, experiment runner, metrics, report
Claude Code 1	Human 1's branch	Generate boilerplate, API wrappers, structured outputs, orchestrator tests
Claude Code 2	Human 2 or 3's branch	Generate sandbox tests, manifest validators, metrics utilities; work on only one branch at a time

There is no frontend owner. For the mid-evaluation, use a CLI. There is no need to build React, a dashboard, or a separate web server.

There is also no database server owner: use JSON/JSONL files and SQLite only if you need indexed queries. Persistent memory can start as JSON records, matching the proposal's scoped-memory design.

Collision-proof working rules

1. Human ownership is authoritative. A Claude Code instance proposes and edits code, but the human reviews and commits it.
2. Each feature branch has one human owner at a time. Do not have multiple people and coding agents editing the same working tree simultaneously.
3. Create a pull request or merge commit into `main` at least twice daily. For this short sprint, the human owner can merge after running tests; formal reviews can be lightweight.
4. Keep shared schemas in a small `schemas.py` or `models.py` module. Agree on them before parallel implementation.
5. Never let a Claude Code instance merge to `main`, rewrite shared history, or delete another person's work without explicit approval.
6. Commit small, working increments. Never wait until Day 5 to integrate.

Suggested structure:

```
testorcode/
├── pyproject.toml
├── README.md
├── .env.example
├── src/testorcode/
│   ├── schemas.py
│   ├── orchestrator.py
│   ├── llm_client.py
│   ├── agents/
│   │   ├── triage.py
│   │   ├── intent_analyst.py
│   │   ├── repairer.py
│   │   └── integrity_guard.py
│   ├── tools/
│   │   ├── pytest_runner.py
│   │   ├── git_tools.py
│   │   └── patch_tools.py
│   ├── retrieval.py
│   ├── memory.py
│   └── evaluation.py
```

```
├── datasets/
│   └── pilot_episodes.jsonl
├── tests/
├── results/
└── scripts/
```

The exact module names can change; the key is that each person owns distinct implementation domains and the interfaces remain stable.

2. Build vs. borrow: what to use and what to implement

The rule for the next five days is simple: borrow commodity infrastructure; implement the research contribution.

Your proposal does not claim to invent a new agent algorithm. Its contribution is the combination of intent-grounded retrieval, scoped memory backed by executable probes, independent test-integrity verification, and a controlled evaluation.

AgentZero.pdf

Recommended stack

Requirement	Use	Why
Language	Python 3.11 or 3.12	Mature testing and AI libraries
Project/dependency setup	`uv` or `pip` + `pyproject.toml`	Fast, reproducible installation
Test execution	`pytest`	Directly matches the proposal
Git history	Python `subprocess` calling Git	Avoid building a Git parser
LLM integration	One provider's SDK or OpenAI-compatible client	Keep a single model interface
Retrieval	`rank-bm25`	Fastest useful first version

Requirement	Use	Why
Embeddings	`sentence-transformers` or a provider embedding API	Add after BM25 works
Structured outputs	Pydantic	Validate decisions and tool arguments
Memory storage	JSONL or JSON	No database setup overhead
Experiment results	JSONL + pandas	Easy to inspect and aggregate
Mutation testing	`mutmut`, or a small cached-mutant runner	Borrow mutation machinery where practical
Sandbox	Docker + Git worktrees	Isolate execution and file modifications
Tests for your own code	`pytest`	Catch integration failures quickly

For your time constraint, I would not introduce LangGraph, CrewAI, AutoGen, a vector database, a frontend framework, or a separate backend server unless the existing team already knows one and has working code. A simple Python state machine is enough to implement the four logical agent roles.

What to borrow versus implement from scratch

Borrow or configure

- LLM API transport, authentication and retry handling.
- BM25 ranking implementation and embedding model inference.
- Pytest execution, Git operations and Docker isolation primitives.
- Mutation generation where a standard tool supports your Python targets.
- Data validation, result aggregation and plotting.

Implement yourselves — these are the project

- The four role-specific responsibilities and their orchestration.
- The `UPDATE_TEST` / `FIX_CODE` / `ESCALATE` decision policy.

- Intent evidence extraction, references and expected-behaviour hypotheses.
- Scoped memory records, executable-probe validation and approval-gated writes.
- Integrity Guard rules and the independent approval/rejection decision.
- The Full/B1/A1/A2/A3 configurations and experiment runner.
- Ground-truth episode construction, metric definitions and failure analysis.

What can be mocked for the mid-evaluation?

- Frontend: omit it entirely.
- Maintainer approval: implement a simple CLI confirmation or a deterministic simulated-maintainer policy that checks the gold label and verification results. Mark it as simulated.
- Memory persistence: use a local JSON file, but execute the probe against the current code before reusing the record.
- Mutation testing: start with a few cached, verified mutants. If mutation execution is not ready, report that part as incomplete and do not claim measured mutant-kill retention.
- Hosted model: you can mock an API response in unit tests, but the main demonstration must include at least one actual model call.
- Embedding retrieval: defer it if BM25 is not yet working. Keep the retrieval interface modular so you can add hybrid retrieval later.

Do not mock the research results themselves. A simulated approval mechanism is an implementation simplification; a fabricated benchmark result is not.

Suggested Python dependencies

Start with a small dependency list, then add packages only when needed:

```
uv init --package testorcode
uv add pytest pydantic python-dotenv rank-bm25 pandas
uv add openai
uv add --dev ruff
```

Use `git` and Docker as system tools. Add `sentence-transformers` or `mutmut` only when the corresponding component is ready.

For safety, the repair agent must not have unrestricted shell access to the host. The host-side orchestrator should validate paths, run allowed commands, enforce resource limits and inspect the patch before it is applied. A prompt saying “do not modify unrelated files” is not a security boundary.

3. Model strategy: hosted API or local hosting?

My recommendation: hosted API for the mid-evaluation

With five days remaining, self-hosting adds another failure surface: model download, quantization, serving, context configuration, inference speed and hardware compatibility.

Your proposal already specifies hosted LLM APIs, with Qwen3 Coder 480B on NVIDIA NIM as its first choice and Mistral Devstral 2 as a backup. It also says to select the final model using the development stream, then freeze its model ID before test-set evaluation.

AgentZero.pdf

For the mid-evaluation, use the first provider that gives you reliable tool calling, structured output and sufficient quota. You can revisit the final model choice after the MVP works.

Option A — Hosted APIs

First try: Gemini Flash through Google AI Studio

Potentially lowest cost

Google's official pricing page lists free-tier access for eligible models, subject to rate limits. The paid Gemini Flash tier is priced per token, and the page warns that free-tier content may be used to improve Google's products.

Why try it: it may let you build the MVP without initial API expenditure. Test actual function calling, JSON outputs and rate limits before committing to it.

Alternative: OpenAI API

The current official pricing page lists GPT-6 Luna at $0.05 per million input tokens and $0.25 per million output tokens in its all-models table. Verify that this model is available in your API account and supports your required tool-calling workflow before selecting it.

OpenAI's published GPT-4.1 mini documentation also lists structured outputs and function calling, but the page marks that model deprecated, so I would not start a new project on it without checking its replacement.

Proposal-aligned option: Qwen Coder through a hosted endpoint

Your proposal already names Qwen3 Coder 480B through NVIDIA NIM. However, NVIDIA's model page currently marks the free endpoint as deprecated and partner endpoints as available.

Therefore, do not build your five-day plan around the assumption that a free Qwen3 Coder 480B endpoint is available. Check endpoint access, credits, rate limits and API compatibility first.

Which is fastest and cheapest?

For your specific MVP, I would test Gemini Flash first if you have access to its free tier. If it fails your tool-calling or reliability test, use the most reliable already-accessible hosted API.

I would not claim a universal speed winner without benchmarking the actual accounts and endpoints available to you. Free-tier quotas, latency and availability can change. For the final research experiment, one frozen model across Full, B1 and all three ablations matters more than choosing the cheapest possible model.

Also, distinguish your Claude Code subscriptions from API access: access to Claude Code does not automatically give your Python application free Claude API calls.

Option B — Self-host an open-source model

If you need a local model for the demo, use Ollama rather than setting up vLLM or a complex serving stack.

The official Qwen repository documents running models through Ollama and accessing the server through an OpenAI-compatible API endpoint.

Here is the practical choice:

Model class	Hardware planning estimate	Recommendation
Qwen3 0.6B, quantized	4–8 GB system RAM; CPU can suffice	Smallest smoke-test model; weak choice for autonomous repair decisions
Qwen3 4B, quantized	8–12 GB system RAM; roughly 4–6 GB VRAM preferred	Small local demo, with limited reliability on complex tool workflows
Qwen3 8B, quantized	16 GB system RAM; roughly 6–8 GB VRAM preferred	Better local prototype if suitable hardware is available

These are approximate planning estimates, not guaranteed minimums. Actual memory depends on quantization, context length, runtime and concurrent requests. Allow extra RAM for the Python application, repository tests and Docker.

Fastest way to try a local model

Install Ollama, then run:

```
ollama run qwen3:4b
```

For the Python application, configure your client to use:

```
Base URL: http://localhost:11434/v1/
Model:    qwen3:4b
```

The exact model identifier must match what Ollama has installed. Run the server before calling the API.

Start with a short context and one request at a time. Do not let all four agents make concurrent requests to a small local model; that can make the demo much

slower.

Would I use local hosting for this project?

Only if you already have a suitable machine and the hosted API options fail.

A 0.6B model may be enough to demonstrate the software plumbing, but that is not the same as demonstrating sound repair decisions. Even a 4B or 8B model may struggle with intent evidence, tool arguments, ambiguous failures and safe patch generation.

More importantly, the submitted proposal expects the same hosted model across the experimental conditions. Switching to a local model just to save API costs would change the proposed compute setup. If you do that, document the deviation and keep the comparison fair.

Final setup decisions

Here is the configuration I would commit to today.

Git: `main` plus three feature branches.

Humans own branches; Claude Code instances edit under supervision. Merge working increments twice daily.

Backend: Python, pytest, Pydantic, Git subprocesses, Docker and a single orchestrator.

No frontend or separate database service.

Retrieval and memory: BM25 first, JSONL episode/result storage and JSON-backed scoped memory with executable probes.

Add embeddings only after the baseline pipeline works.

Model: test an eligible Gemini Flash endpoint first; use a reliable hosted alternative if necessary.

Use Ollama with a small Qwen3 model only as a local fallback, not as a substitute for research-quality validation.

Research: implement Full, B1, A1, A2 and A3 as configuration switches.

Run a small, labelled pilot now; reserve the full evaluation matrix for the remaining 15 days.

The single most important action for today: agree on the episode and result schemas, create the three feature branches, and get one real model call plus one real pytest run working before assigning large implementation tasks. Those two working foundations will tell you whether the rest of the five-day plan is on track.

IMP POINTS FROM ABOVE RESPONSES:

RULES TO FOLLOW:

1. use different branches and for each use different claude instances.
    a. agent-core - human1 + claudecode1
    b. sandbox - human2 + claudecode2
    c. evaluation - human3 + claudecode2
2. Use one episode format and one result format across all five experimental conditions.
    a. finalised by human1
3. Each run should record the condition, seed, model ID, predicted labels, retrieved evidence, patch, final test status, integrity results, token usage, tool calls and error details.
    a. taken care by all
4. Do not expose the gold labels or reference repair to the agent itself. Keep them in the evaluation harness.
    a. taken care by human2, human3
5. the hosted model should be accessed through a common client, and all conditions must use the same model and comparable budgets.
    a. taken care by human1

Rules for the next five days

1. Working code beats elegant architecture: Prefer a deterministic Python state machine with structured agent calls over a new framework.
2. No frontend: A CLI and a clear terminal demonstration are sufficient.
3. No optional fine-tuning: Your proposal already identifies it as the first feature to drop when time is short.
4. No separate applications for ablations: Use configuration flags to switch components off.
5. No unverified benchmark claims: Every number in the report must come from logged experiments.
6. Commit every working milestone: Never wait until Day 5 to push the code.
7. Freeze features on Day 4 and freeze the release on Day 5. Only bug fixes after that.

Aim to show three things:

1. a working end-to-end MVP,
2. real Full-versus-B1 and preliminary ablation results,
3. a GitHub repository plus report that makes the remaining 15 days look like a credible completion plan rather than an unstarted project.

PROJECT DIRECTORY STRUCTURE

```
testorcode/
├── pyproject.toml
├── README.md
├── .env.example
├── src/testorcode/
│   ├── schemas.py
│   ├── orchestrator.py
│   ├── llm_client.py
│   ├── agents/
│   │   ├── triage.py
│   │   ├── intent_analyst.py
```

```
│   │   ├── repairer.py
│   │   └── integrity_guard.py
│   ├── tools/
│   │   ├── pytest_runner.py
│   │   ├── git_tools.py
│   │   └── patch_tools.py
│   ├── retrieval.py
│   ├── memory.py
│   └── evaluation.py
├── datasets/
│   └── pilot_episodes.jsonl
├── tests/
├── results/
└── scripts/
```

RECOMMENDED STACK

Requirement	Use	Why
Language	Python 3.11 or 3.12	Mature testing and AI libraries
Project/dependency setup	`uv` or `pip` + `pyproject.toml`	Fast, reproducible installation
Test execution	`pytest`	Directly matches the proposal
Git history	Python `subprocess` calling Git	Avoid building a Git parser
LLM integration	One provider's SDK or OpenAI-compatible client	Keep a single model interface
Retrieval	`rank-bm25`	Fastest useful first version
Embeddings	`sentence-transformers` or a provider embedding API	Add after BM25 works
Structured outputs	Pydantic	Validate decisions and tool arguments
Memory storage	JSONL or JSON	No database setup overhead

Requirement	Use	Why
Experiment results	JSONL + pandas	Easy to inspect and aggregate
Mutation testing	`mutmut`, or a small cached-mutant runner	Borrow mutation machinery where practical
Sandbox	Docker + Git worktrees	Isolate execution and file modifications
Tests for your own code	`pytest`	Catch integration failures quickly

What to borrow versus implement from scratch

Borrow or configure

- LLM API transport, authentication and retry handling.
- BM25 ranking implementation and embedding model inference.
- Pytest execution, Git operations and Docker isolation primitives.
- Mutation generation where a standard tool supports your Python targets.
- Data validation, result aggregation and plotting.

Implement yourselves — these are the project

- The four role-specific responsibilities and their orchestration.
- The `UPDATE_TEST` / `FIX_CODE` / `ESCALATE` decision policy.
- Intent evidence extraction, references and expected-behaviour hypotheses.
- Scoped memory records, executable-probe validation and approval-gated writes.
- Integrity Guard rules and the independent approval/rejection decision.
- The Full/B1/A1/A2/A3 configurations and experiment runner.
- Ground-truth episode construction, metric definitions and failure analysis.

MODEL STRATEGY

- hosted API for the mid-evaluation
    - use gemini flash for building the MVP, later replace this with the better option